

SECUREPULSE: A REAL-TIME MOBILE CYBERSECURITY OPERATIONS & INCIDENT TRIAGE PLATFORM

****TECHNICAL & ACADEMIC PROJECT REPORT****

Comprehensive Engineering Design, Architecture, Implementation, and Verification Dossier

- **Author / Principal Engineer:** Natto Muni Chakma
 - **Technology Stack:** Flutter 3.x • Dart 3.x • FastAPI • WebSockets • Riverpod 2.6 • Render Cloud
 - **Target Platforms:** Android (Primary / Release 57.1MB), iOS, Web
 - **Repository:** NATTOMR/securepulse-mobile
 - **Version:** 1.0.0+1
 - **Date of Publication:** September 2026
-

DECLARATION

I hereby declare that this project report entitled "**SecurePulse: A Real-Time Mobile Cybersecurity Operations & Incident Triage Platform**" is an authentic record of the architectural design, engineering development, implementation, and empirical verification conducted on the SecurePulse platform. All external open-source frameworks, algorithms, protocols, and referenced publications have been cited and acknowledged in full compliance with academic and professional standards.

Signature: *Natto Muni Chakma*

Date: September 1, 2026

ACKNOWLEDGEMENT

Sincere gratitude is extended to the global open-source software engineering and cybersecurity communities whose toolchains formed the foundation of this platform. In particular, appreciation is given to the engineering teams behind **Flutter & Dart**, **FastAPI**, **Wazuh Open Source SIEM**, **Semgrep SAST**, **Riverpod**, and **Google Material Design 3**. Special recognition is also extended to the incident responders and DevSecOps practitioners whose operational workflows inspired the mobile-first triage paradigm of SecurePulse.

ABSTRACT

Modern enterprise cybersecurity operations require continuous visibility, low-latency alerting, and rapid incident triage across heterogeneous cloud and on-premise infrastructure. Traditional Security Operations Center (SOC) workflows remain predominantly tethered to desktop-based SIEM monitors, introducing critical response latency when security analysts and incident commanders are away from their workstations.

SecurePulse is an enterprise-grade mobile cybersecurity platform engineered to eliminate this operational bottleneck. Built with Flutter 3.x and Dart 3.x, SecurePulse interfaces with an asynchronous FastAPI backend to deliver real-time intrusion telemetry, on-demand static application security testing (SAST) repository auditing, conversational AI-assisted vulnerability remediation, and cryptographically stamped PDF compliance report generation directly to mobile devices.

The platform is designed around a **Dual Operation Architecture**: a deterministic, zero-latency **Demo / Offline Simulation Mode** for standalone evaluation and an encrypted **Live API Mode** connecting via REST and WebSockets to cloud infrastructure on Render. The mobile client incorporates hardware-backed biometric authentication (`local_auth`), AES-256 KeyStore credential encryption (`flutter_secure_storage`), persistent offline mutation caching (`hive_flutter`), and full-duplex WebSocket event streaming with automatic HTTP long-polling failover. Verified against a comprehensive 33-case automated test suite (100% pass rate) and compiled with ProGuard/R8 shrinking into an

optimized 57.1 MB release package, SecurePulse provides security teams with an autonomous, hardened, and highly responsive mobile SOC console.

TABLE OF CONTENTS

- **DECLARATION & ACKNOWLEDGEMENT** i
- **ABSTRACT** ii
- **LIST OF FIGURES & TABLES** iii
- **LIST OF ABBREVIATIONS** iv
- **CHAPTER 1 — INTRODUCTION & PROBLEM FORMULATION** 1
 - 1.1 Background and Problem Statement
 - 1.2 Project Aims and Technical Objectives
 - 1.3 System Scope and Target User Personas
 - 1.4 Key Technical Contributions
 - 1.5 Organization of the Report
- **CHAPTER 2 — SYSTEM ARCHITECTURE & DATA FLOW** 4
 - 2.1 Overall System Topology & Architecture Flowchart
 - 2.2 Mobile Clean Architecture & Riverpod State Model
 - 2.3 Cloud Microservice & Asynchronous WebSocket Threat Engine
 - 2.4 Dual Operation Architecture (Demo Simulation vs Live Cloud Mode)
- **CHAPTER 3 — SECURITY & CRYPTOGRAPHIC DESIGN** 8
 - 3.1 Mobile Hardware Security (Biometrics & AES-256 KeyStore Encryption)
 - 3.2 Authentication, Authorization & Stateless JWT Lifecycle
 - 3.3 Dynamic SHA-256 Cryptographic PDF Audit Stamping
 - 3.4 Threat Modeling & STRIDE Security Controls
- **CHAPTER 4 — SYSTEM IMPLEMENTATION & TOOLCHAIN INTEGRATION** ... 12
 - 4.1 Flutter Mobile Client Implementation (Dashboard, Alerts, Repositories)
 - 4.2 Conversational AI Security Copilot & CVE Playbook Engine
 - 4.3 Wazuh SIEM Connector & Manager API Integration
 - 4.4 Semgrep SAST Vulnerability Ingestion & GitHub OAuth2 PKCE Sync
 - 4.5 Offline Action & Mutation Queue Engine

- **CHAPTER 5 — TESTING, VERIFICATION & EMPIRICAL RESULTS** 20
 - 5.1 Comprehensive Automated Test Suite (33 Test Cases)
 - 5.2 Network Contract & Mode Isolation Verification
 - 5.3 WebSocket Failover & Reconnection Resilience
 - 5.4 Static Analysis & Code Quality Results
- **CHAPTER 6 — CLOUD DEPLOYMENT & RELEASE ENGINEERING** 24
 - 6.1 Cloud Microservice Deployment (FastAPI on Render Cloud)
 - 6.2 Android Production Compilation & ProGuard / R8 Optimization
 - 6.3 Google Play Store Packaging & Keystore Signing Configuration
- **CHAPTER 7 — LIMITATIONS, FUTURE ROADMAP & CONCLUSION** 27
 - 7.1 Technical Constraints & Operational Limitations
 - 7.2 Evolutionary Roadmap & Future SOAR Automation
- 7.3 Concluding Remarks
- **REFERENCES & APPENDICES** 29
 - Appendix A: REST & WebSocket API Endpoint Specifications
 - Appendix B: References & Standards Bibliography

LIST OF FIGURES & TABLES

- **Figure 2.1:** Master End-to-End System Architecture Flowchart
- **Figure 2.2:** Mobile Clean Architecture & Riverpod 2.6 State Management Flow
- **Figure 3.1:** Cryptographic Non-Repudiation PDF Digest Stamping Pipeline
- **Figure 4.1:** SecurePulse Mobile User Interface Screens (Dashboard, Codebases, Alerts, AI Copilot)
- **Figure 4.2:** Wazuh SIEM Host Intrusion & Syslog Triage Modal
- **Figure 4.3:** Semgrep SAST Line-Level Vulnerability & Code Highlighting View
- **Figure 5.1:** Automated Test Execution Console Output (33 / 33 Tests Passed)
- **Figure 6.1:** Production Android Release Build (`app-release.apk` , 57.1 MB)
- **Table 1.1:** User Persona Matrix & Operational Requirements
- **Table 2.1:** Dual Mode Operational Contract Comparison Matrix
- **Table 3.1:** STRIDE Threat Analysis & Implemented Countermeasures
- **Table 5.1:** Complete Automated Test Suite Execution Summary (33 Tests)

- **Table 6.1:** Release Package Size Optimization & ProGuard Metrics
-

LIST OF ABBREVIATIONS

- **AES:** Advanced Encryption Standard
 - **API:** Application Programming Interface
 - **ASGI:** Asynchronous Server Gateway Interface
 - **CVE:** Common Vulnerabilities and Exposures
 - **CVSS:** Common Vulnerability Scoring System
 - **CWE:** Common Weakness Enumeration
 - **FIM:** File Integrity Monitoring
 - **HIDS:** Host-based Intrusion Detection System
 - **JWT:** JSON Web Token
 - **PKCE:** Proof Key for Code Exchange
 - **RBAC:** Role-Based Access Control
 - **REST:** Representational State Transfer
 - **SAST:** Static Application Security Testing
 - **SIEM:** Security Information and Event Management
 - **SOAR:** Security Orchestration, Automation, and Response
 - **SOC:** Security Operations Center
 - **WSS:** WebSocket Secure
-

CHAPTER 1 — INTRODUCTION & PROBLEM FORMULATION

1.1 Background and Problem Statement

In modern enterprise environments, enterprise networks, containerized clusters, and multi-cloud environments generate thousands of security alerts daily. Security Operations Centers (SOCs) rely on Security Information and Event Management (SIEM) systems (such as Wazuh, Splunk, Elastic) and Static Application Security Testing (SAST) engines (such as Semgrep) to identify threats and vulnerabilities.

However, traditional incident response remains almost exclusively desktop-centric. When on-call security engineers, CISOs, or incident commanders are away from their workstations, their ability to inspect intrusion alerts, review vulnerability details, dispatch mitigation commands (such as IP quarantining or agent restarts), and verify compliance posture is severely impaired. Email and push notifications provide passive text alerts without the interactive triage, raw syslog inspection, or cryptographic reporting required for rapid remediation.

1.2 Project Aims and Technical Objectives

The primary objective of **SecurePulse** is to design, implement, and verify a hardened, mobile-first cybersecurity operations console that empowers security practitioners to monitor, triage, and remediate security events from any location.

The specific engineering objectives are:

- **Low-Latency Mobile Telemetry:** Deliver a responsive mobile client built with Flutter 3.x featuring a real-time WebSocket threat stream (`/ws/alerts`) with automatic fallback to REST polling.
- **Multi-Source Toolchain Integration:** Interface natively with Wazuh HIDS for intrusion telemetry and Semgrep SAST for code vulnerability auditing.
- **Conversational AI Incident Remediation:** Integrate an AI security assistant providing immediate, actionable patch guidance for high-impact CVEs (e.g., CVE-2024-3094, SQL injection).
- **Hardware-Backed Mobile Hardening:** Enforce biometric app locking (`local_auth`), AES-256 KeyStore token storage (`flutter_secure_storage`), and zero hardcoded secrets.
- **Cryptographic Compliance Stamping:** Implement an on-device pure-Dart PDF engine computing deterministic SHA-256 audit digests for SOC 2, ISO 27001, PCI-DSS, and HIPAA reports.

- **Dual Mode Operational Resilience:** Support both an offline, deterministic **Demo Simulation Mode** and a production **Live Cloud Mode** with an offline action/mutation queue (`hive_flutter`).

1.3 System Scope and Target User Personas

SecurePulse is targeted at three distinct operational roles within an enterprise:

- **SOC Tier 1/2 Analysts:** Real-time triage of incoming intrusion alarms, raw syslog analysis, and one-tap IP quarantine actions.
- **DevSecOps Engineers & Tech Leads:** Monitoring code repository security grades (A–F), tracking line-level SAST findings, and triggering on-demand scans.
- **CISOs & Compliance Auditors:** Reviewing organizational posture score gauges (0–100) and generating cryptographically signed compliance audit reports.

1.4 Key Technical Contributions

- **Dual Mode Abstraction Layer:** Seamless switching between offline simulated telemetry and live cloud API backends without code divergence.
 - **Offline Mutation Queue Engine:** Durable Hive-backed queueing of mitigation actions when disconnected, with automatic replay upon network recovery.
 - **Cryptographic Evidence Hashing:** Real-time SHA-256 hash generation guaranteeing non-repudiation and tamper-evidence on mobile-generated compliance dossiers.
 - **Optimized Android Distribution Bundle:** Complete ProGuard/R8 configuration reducing application package footprint to 57.1 MB.
-

CHAPTER 2 — SYSTEM ARCHITECTURE & DATA FLOW

2.1 Overall System Topology & Architecture Flowchart

The SecurePulse ecosystem connects the mobile presentation tier, the cloud backend service tier, external security data sources, and cloud database infrastructure in a clean, decoupled topology:

flowchart TB

S["SECUREPULSE PLATFORM"]

S --> M[" MOBILE CLIENT (Flutter 3.x)"]
S --> B[" CLOUD BACKEND (FastAPI ASGI)"]
S --> SS[" SECURITY SOURCES (Wazuh / Semgrep)"]
S --> P[" PROCESSING & SOAR PIPELINE"]
S --> AI[" ARTIFICIAL INTELLIGENCE"]
S --> I[" INFRASTRUCTURE (Render / Postgres)"]
S --> SEC[" SECURITY & CRYPTOGRAPHY"]

M --> D["Dashboard (Posture Gauge 88%)"]
M --> R["Repositories (SAST Auditing)"]
M --> A["Alerts (SIEM Intrusion Feed)"]
M --> C["AI Copilot (CVE Remediation)"]
M --> ST["Settings (Environment Switcher)"]

B --> F["FastAPI Async Engine"]
B --> API["REST API (/v1/*)"]
B --> WS["WebSocket Engine (/ws/alerts)"]
B --> DB["Database Tier (PostgreSQL / Hive)"]

SS --> W["Wazuh SIEM / HIDS"]
SS --> G["GitHub Repository Telemetry"]
SS --> SEM["Semgrep SAST Scanner"]

W --> EV["Security Event Ingestion"]
G --> EV
SEM --> EV

EV --> P
P --> FIND["Vulnerability Findings"]
P --> ALERT["SOC Threat Alerts"]
P --> SCAN["On-Demand SAST Scans"]
P --> RISK["Composite Posture Engine"]

F --> API
F --> WS

```

F --> DB

WS --> A
WS --> D

API --> D
API --> R
API --> A
API --> C

AI --> C
AI --> LLM["Cloud LLM Proxy (Live)"]
AI --> OFF["Offline Heuristic Engine"]

I --> REN["Render Cloud Web Service"]
I --> PG["Managed PostgreSQL 16 (SSL)"]
I --> HTTPS["TLS 1.3 Transport (HTTPS/WSS)"]

SEC --> JWT["Stateless JWT Authentication"]
SEC --> RBAC["Role-Based Access Control"]
SEC --> SECRET["AES-256 Android KeyStore"]
SEC --> WEB["HMAC-SHA256 Signatures"]
SEC --> AUDIT["SHA-256 PDF Audit Stamping"]

REN --> F
PG --> DB
HTTPS --> WS

classDef main fill:#0b1220,stroke:#2563eb,color:#ffffff,stroke-width:3px;
classDef mobile fill:#111827,stroke:#3b82f6,color:#ffffff;
classDef backend fill:#111827,stroke:#22c55e,color:#ffffff;
classDef security fill:#111827,stroke:#ef4444,color:#ffffff;
classDef ai fill:#111827,stroke:#a855f7,color:#ffffff;

class S main;
class M,D,R,A,C,ST mobile;
class B,F,API,WS,DB backend;
class SEC,JWT,RBAC,SECRET,WEB,AUDIT security;
class AI,LLM,OFF ai;

```

2.2 Mobile Clean Architecture & Riverpod State Model

The mobile client strictly adheres to **Clean Architecture** principles, partitioned into three concentric layers:

- **Presentation Layer:** Flutter widgets, theme tokens (`AppTheme`), declarative GoRouter navigation (`app_router.dart`), and reactive Riverpod providers (`app_providers.dart`).
- **Domain Layer:** Pure Dart entities and data contracts (`AlertModel` , `RepositoryModel` , `WazuhAgentModel` , `FindingModel` , `UserModel`).

- **Data & Infrastructure Layer:** Concrete repository implementations (`AlertsRepositoryImpl` , `WazuhRepositoryImpl` , `AuthRepositoryImpl`), Dio-based HTTP network client (`ApiClient`), and local storage wrappers (`SecureStorageService` , `HiveStorageService` , `OfflineQueueService`).

2.3 Cloud Microservice & Asynchronous WebSocket Engine

The backend tier is built using **FastAPI** (Python 3.11 ASGI). It exposes REST endpoints protected by JWT Bearer authentication and provides a bidirectional WebSocket endpoint at `/ws/alerts` . The WebSocket service automatically translates URL schemes (`http://` → `ws://` , `https://` → `wss://`), validates authentication query tokens during handshake, and broadcasts structured incident frames to connected mobile clients.

2.4 Dual Operation Architecture

Metric / Feature	Demo / Standalone Mode	Live Cloud Backend Mode
Backend Dependency	None (Fully autonomous on-device)	FastAPI on Render Cloud
Data Ingestion	Deterministic high-fidelity security mocks	Real-time REST & <code>/ws/alerts</code> WebSocket
Authentication	Local Instant Session / Biometric Bypass	JWT HS256 Token with AES-256 Storage
Offline Resilience	Zero latency (100% offline cache)	Encrypted Hive caching + Offline Mutation Queue
AI Copilot	Offline CVE-2024-3094 & SQLi playbooks	Live Cloud LLM Streaming Proxy

CHAPTER 3 — SECURITY & CRYPTOGRAPHIC DESIGN

3.1 Mobile Hardware Security & KeyStore Encryption

All sensitive data (JWT access tokens, environment URLs, user credentials) are encrypted at rest using the device's hardware security module:

- **Android:** Hardware-backed Android KeyStore generating AES-256 GCM master encryption keys.
- **iOS:** Apple Keychain Services with `kSecAttrAccessibleAfterFirstUnlock`.
- **Biometric Authentication:** Enforced via `local_auth` (`FlutterFragmentActivity` on Android) requiring fingerprint or biometric face confirmation before session initialization.

3.2 Authentication & Stateless JWT Lifecycle

Authentication follows the **OAuth 2.0 / RFC 7519 JWT standard**. Upon successful credential verification or GitHub OAuth2 PKCE callback, the server issues a digitally signed JWT containing user ID, role (`analyst`, `admin`, `auditor`), and expiration timestamp. The `ApiClient` automatically injects this token as a `Bearer` HTTP header via a custom Dio interceptor and purges it from hardware storage on user logout.

3.3 Dynamic SHA-256 Cryptographic Audit Stamping

To ensure tamper-evidence on generated compliance reports (SOC 2, ISO 27001, PCI-DSS, HIPAA), `[ ReportPdfService ](file:///e:/SOC%20projects/securepulse-mobile/lib/core/services/report_pdf_service.dart)` implements an on-device cryptographic hashing algorithm:

$$\text{Digest} = \text{SHA-256}(\text{AUDIT_ID} \parallel \text{TIMESTAMP} \parallel \text{FRAMEWORK} \parallel \text{SCORE} \parallel \text{sum}(\text{ControlFindings}))$$

The resulting 64-character hexadecimal digest is embedded directly into the Auditor Sign-off box on the PDF. Any subsequent alteration of compliance metrics produces a cryptographic avalanche effect, invalidating the verification fingerprint.

3.4 Threat Modeling & STRIDE Controls

Threat Category	Potential Attack Vector	Implemented Security Control
Spoofing	Forged WebSocket threat broadcasts	Handshake JWT token query validation & origin verification.
Tampering	Modification of compliance audit findings	Deterministic SHA-256 cryptographic digest stamped on exported PDFs.
Repudiation	Denied incident mitigation actions	Immutable action logging with user ID and timestamp attribution.
Information Disclosure	Memory sniffing of auth tokens	AES-256 KeyStore storage and explicit memory clearing on logout.
Denial of Service	Network connection loss during mitigation	Offline Action Queue caching mutations and auto-flushing on reconnect.
Elevation of Privilege	Unauthorized API status updates	Role-Based Access Control (RBAC) enforced on backend routes.

CHAPTER 4 — SYSTEM IMPLEMENTATION & TOOLCHAIN INTEGRATION

4.1 Flutter Mobile Client Implementation

The mobile user interface is engineered in Flutter 3.x using **Google Material Design 3** and custom Cyber Dark Obsidian (#0A0E1A) / Clean Light (#F8FAFC) palettes:

- **Executive Dashboard:** Renders a circular animated Posture Gauge (88/100), active incident cards, service health latency badges, and an interactive `f1_chart` vulnerability donut chart.
- **Monitored Repositories Screen:** Ingests codebases, branch metadata, last scan timestamps, and A–F security grade badges.
- **SOC SIEM Alert Feed:** Displays real-time intrusion alarms with severity filter pills (`[All]` , `[Critical]` , `[High]` , `[Medium]` , `[Low]`) and detailed triage modal dialogs.

4.2 Conversational AI Security Copilot

The AI Security Copilot (`AiAssistantScreen`) provides interactive threat analysis and patch suggestions. In Demo Mode, an offline heuristic engine parses security queries and delivers complete remediation scripts for major vulnerabilities (such as CVE-2024-3094 in XZ Utils and SQL injection in Python/FastAPI). In Live Mode, queries are proxied to an upstream cloud LLM.

4.3 Wazuh SIEM Connector & Manager API

Built inside `[ WazuhRepositoryImpl ]`(`file:///e:/SOC%20projects/securepulse-mobile/lib/features/alerts/data/wazuh_repository.dart`):

- **Agent Inventory:** Ingests connected Wazuh agents (`wazuhAgentModel`) across Linux (Ubuntu, RHEL, Debian) and Windows Server with OS versions, IP addresses, and keepalive status.
- **Cluster Daemon Probes:** Monitors operational health for core Wazuh daemons (`wazuh-analysisd` , `wazuh-remoted` , `wazuh-modulesd` , `wazuh-authd` , `wazuh-db`).
- **Remote Management:** Supports remote agent and daemon restart dispatches via `POST /v1/wazuh/agents/{id}/restart` .

4.4 Semgrep SAST Vulnerability Ingestion & GitHub OAuth2 Sync

- **Semgrep SAST Integration:** Normalizes static analysis findings, attributing vulnerabilities to specific code files and line numbers (e.g. `app/services/auth_service.py:84`) with CWE categorization (CWE-89, CWE-798).
- **GitHub OAuth2 PKCE Flow:** Implemented in [`GithubOAuthService`] (file:///e:/SOC%20projects/securepulse-mobile/lib/core/services/github_oauth_service.dart). Generates 256-bit cryptographic CSRF state tokens, opens the system browser, captures deep-link callbacks (`securepulse://oauth/callback`), and exchanges authorization codes for JWT session tokens.

4.5 Offline Action & Mutation Queue Engine

Implemented in [`OfflineQueueService`](file:///e:/SOC%20projects/securepulse-mobile/lib/core/services/offline_queue_service.dart):

- **Durability:** Captures mitigation actions (alert status changes, scan dispatches) into local encrypted Hive storage when offline.
 - **Automatic Replay:** Subscribes to `connectivity_plus` network events and automatically flushes queued mutations via `ApiClient` upon network restoration.
 - **Poison Queue Protection:** Tracks mutation retry counts and drops requests failing more than 5 times.
-

CHAPTER 5 — TESTING, VERIFICATION & EMPIRICAL RESULTS

5.1 Comprehensive Automated Test Suite

The SecurePulse platform includes an automated unit and integration test suite executed via `flutter test`. All **33 / 33 tests pass with a 100% success rate (0 errors, 0 skips)**.

```
00:00 +0: ApiClient initializes with correct default headers and base URL ..... PASS
00:00 +1: Auth token header is correctly set and cleared ..... PASS
00:00 +2: Base URL can be dynamically updated for environment switching ..... PASS
00:00 +3: ApiEndpoints contract verification ..... PASS
00:00 +4: ApiException error classification checks ..... PASS
00:00 +5: AuthRepository returns demo user when isDemoMode is true ..... PASS
00:00 +6: DashboardRepository returns structured demo telemetry ..... PASS
00:00 +7: RepositoryRepository returns mock codebases when isDemoMode ..... PASS
00:00 +8: AlertsRepository returns mock SIEM incidents when isDemoMode ..... PASS
00:00 +9: AiRepository generates local security advice when isDemoMode ..... PASS
00:00 +10: AuthRepository loginWithGitHub returns demo user when isDemoMode ..... PASS
00:00 +11: Live API mode is active when isDemoMode is false ..... PASS
00:00 +12: WebSocketService correctly converts http to ws for local emulator ..... PASS
00:00 +13: WebSocketService correctly converts https to wss for Render cloud ..... PASS
00:00 +14: WebSocketService stays disconnected when Demo Mode is active ..... PASS
00:00 +15: GithubOAuthService generates high-entropy random state ..... PASS
00:00 +16: GithubOAuthService builds well-formed authorization URI ..... PASS
00:00 +17: GithubOAuthService extracts valid code from callback URI ..... PASS
00:00 +18: GithubOAuthService rejects callback URI with mismatched state (CSRF) ..... PASS
00:00 +19: GithubOAuthService ignores unrelated deep links gracefully ..... PASS
00:00 +20: ReportPdfService computes deterministic 64-char hex SHA-256 digest ..... PASS
00:00 +21: Audit digest exhibits cryptographic avalanche effect on data change ..... PASS
00:00 +22: ReportPdfService builds valid PDF document bytes with %PDF header ..... PASS
00:00 +23: WazuhRepository returns structured agent inventory in Demo Mode ..... PASS
00:00 +24: WazuhRepository returns cluster daemon statuses in Demo Mode ..... PASS
00:00 +25: WazuhRepository restartAgent and restartDaemon actions succeed ..... PASS
00:00 +26: WazuhAgentModel serialization and deserialization validation ..... PASS
00:00 +27: WazuhDaemonModel serialization and deserialization validation ..... PASS
00:00 +28: OfflineMutation model serialization and deserialization ..... PASS
00:00 +29: OfflineQueueService enqueues, queries, and removes mutations ..... PASS
00:00 +30: OfflineQueueService flushQueue handles empty queue gracefully ..... PASS
00:00 +31: OfflineMutation copyWith creates modified clones correctly ..... PASS
00:02 +32: SecurePulse Mobile app pump test ..... PASS
```

Total Results: 33 Passed, 0 Failed, 0 Skipped (100% Pass Rate)

5.2 Static Analysis Verification

Execution of `dart analyze` across the entire codebase completed with **0 issues found**, validating strict adherence to Dart formatting, null-safety guarantees, and zero unused dependencies.

CHAPTER 6 — CLOUD DEPLOYMENT & RELEASE ENGINEERING

6.1 Cloud Microservice Deployment

The FastAPI backend service is deployed on **Render Cloud** as a containerized web service (<https://secureguard-backend-7eqm.onrender.com>). It is backed by a managed PostgreSQL 16 database instance enforcing SSL connections. The deployment features automated Git CI/CD triggering on push to the `main` branch.

6.2 Android Production Packaging & ProGuard / R8 Optimization

The Android production build was compiled using `flutter build apk --release`. ProGuard and R8 shrinking rules in `[ android/app/proguard-rules.pro ]`(`file:///e:/SOC%20projects/securepulse-mobile/android/app/proguard-rules.pro`) stripped unused code while preserving critical reflection classes for Flutter embedding, KeyStore cryptography, and BiometricPrompt.

- **Original Uncompressed Package Size:** 63.6 MB
- **Optimized Release Package Size (`app-release.apk`):** **57.1 MB** (10.2% reduction)
- **Build Time:** 130.7s

6.3 Google Play Store Signing Configuration

The build environment is configured with `signingConfigs.release` loading credentials from `key.properties`. A template is provided in `[ android/key.properties.example ]` (`file:///e:/SOC%20projects/securepulse-mobile/android/key.properties.example`) supporting 2048-bit RSA keys generated via `keytool` for Android App Bundle (AAB) commercial distribution.

CHAPTER 7 — LIMITATIONS, FUTURE ROADMAP & CONCLUSION

7.1 Technical Constraints & Operational Limitations

- **Push Notifications:** Remote cloud push notifications require supplying a registered project `google-services.json`. In its absence, the app falls back safely to in-memory event streams without crashing.
- **Edge Execution:** Threat mitigations (IP quarantine) are dispatched to backend event listeners rather than modifying edge firewalls directly from the mobile client.

7.2 Evolutionary Roadmap & Future SOAR Automation

- **Autonomous AI Remediation:** Generating automated GitHub Pull Request patches directly from AI Copilot recommendations.
- **Multi-Cloud SOAR Connectors:** Native integrations with AWS Network ACLs, Cloudflare WAF, and Cortex XSOAR.
- **Multi-Tenant MSSP Portal:** Dynamic switching between client organizations with tenant-scoped cryptographic keys.
- **On-Device Small Language Models (SLMs):** Quantized Gemma-2B / Llama-3-1B execution via ONNX Runtime for fully offline AI triage.

7.3 Concluding Remarks

SecurePulse successfully bridges the gap between enterprise security monitoring and mobile operational mobility. By unifying Wazuh SIEM telemetry, Semgrep SAST auditing, conversational AI assistance, and SHA-256 cryptographic audit stamping into an encrypted, biometrically secured Flutter client, the platform delivers a comprehensive, production-ready mobile SOC console.

REFERENCES & APPENDICES

Appendix A: REST & WebSocket API Endpoint Specifications

- `GET /health` — Service uptime probe and database health check.
- `POST /v1/auth/login` — Analyst credential exchange returning JWT bearer token.
- `POST /v1/auth/github` — OAuth2 PKCE authorization code exchange.
- `GET /v1/dashboard/summary` — Posture gauge score and severity telemetry counts.
- `GET /v1/repositories` — Monitored codebase inventory with health grades.
- `POST /v1/repositories/{id}/scan` — On-demand SAST scan trigger dispatch.
- `GET /v1/soc/alerts` — Paginated SOC intrusion alert stream.
- `PUT /v1/soc/alerts/{id}/status` — Alert lifecycle mutation (`open` → `resolved`).
- `GET /v1/wazuh/agents` — Wazuh connected agent inventory inspection.
- `POST /v1/wazuh/agents/{id}/restart` — Remote Wazuh agent restart command.
- `WS /ws/alerts` — Full-duplex WebSocket threat incident stream.

Appendix B: Standards & Technical References

- **NIST SP 800-61 Rev. 2:** *Computer Security Incident Handling Guide*, National Institute of Standards and Technology.
- **RFC 7519:** *JSON Web Token (JWT)*, Internet Engineering Task Force (IETF).
- **OWASP Mobile Top 10 (2024):** *Mobile Application Security Verification Standard (MASVS)*.
- **Wazuh Documentation:** *Open Source Security Platform Architecture & HIDS Ruleset*, Wazuh Inc.
- **Semgrep OSS Documentation:** *Static Analysis Engine & Rule Syntax*, Return To Corporation.
- **Flutter Framework Documentation:** *Clean Architecture and State Management with Riverpod*, Google LLC.